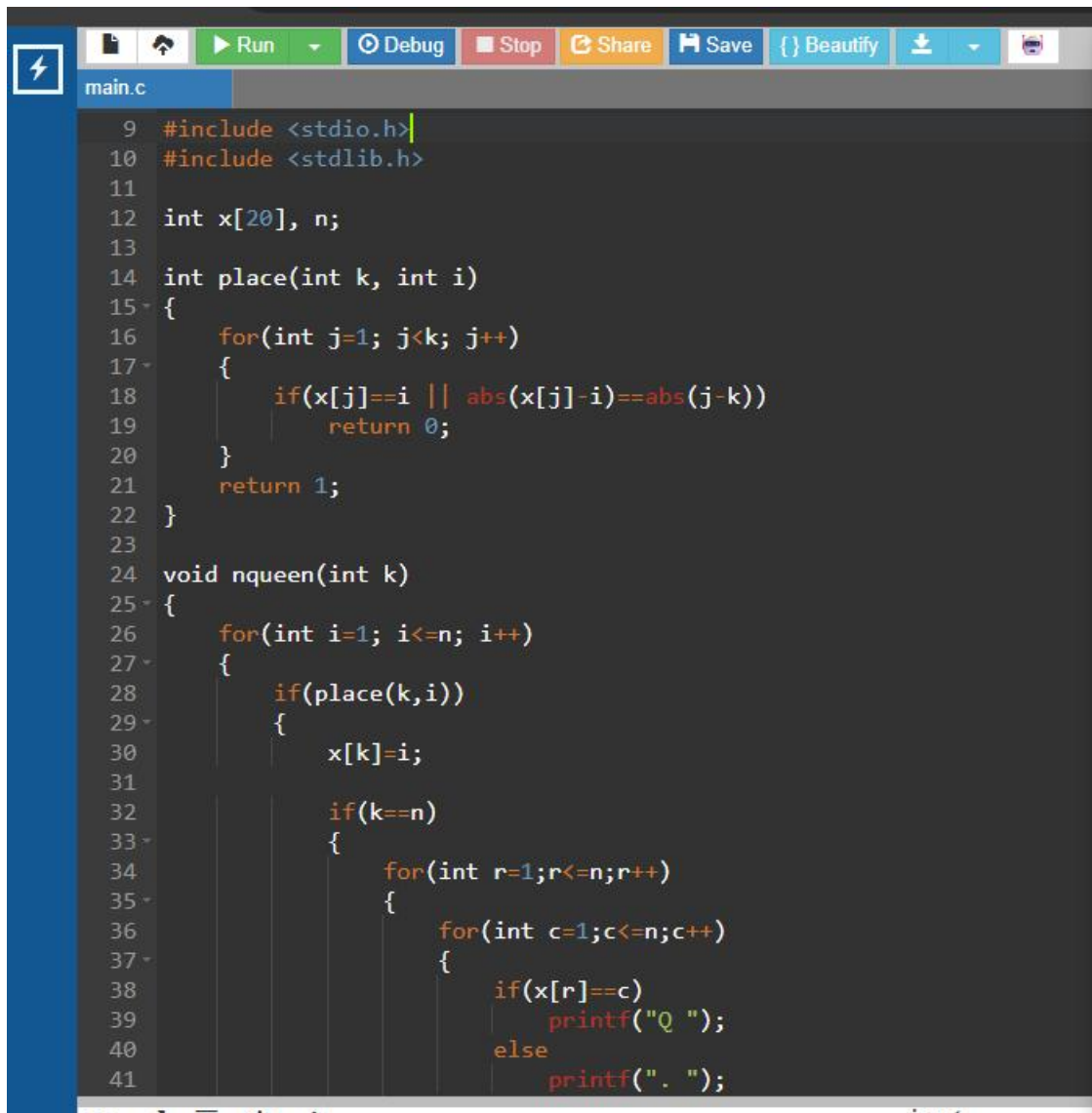


N-Queens, Sudoku puzzle, combinations, permutations

1. Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions.
 - a. Visualization for 4-Queens:
Input: N = 4
Output:
Explanation: Each 'Q' represents a queen, and '.' represents an empty space.
 - b. Visualization for 5-Queens:
Input: N = 5
Output:
 - c. Visualization for 8-Queens:
Input: N = 8
Output:



```
main.c
9  #include <stdio.h>
10 #include <stdlib.h>
11
12 int x[20], n;
13
14 int place(int k, int i)
15 {
16     for(int j=1; j<k; j++)
17     {
18         if(x[j]==i || abs(x[j]-i)==abs(j-k))
19             return 0;
20     }
21     return 1;
22 }
23
24 void nqueen(int k)
25 {
26     for(int i=1; i<=n; i++)
27     {
28         if(place(k,i))
29         {
30             x[k]=i;
31
32             if(k==n)
33             {
34                 for(int r=1; r<=n; r++)
35                 {
36                     for(int c=1; c<=n; c++)
37                     {
38                         if(x[r]==c)
39                             printf("Q ");
40                         else
41                             printf(". ");
```

```
Enter N: 4
```

```
. Q . .  
. . . Q  
Q . . .  
. . Q .  
  
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.
 - a. 8×10 Board:
8 rows and 10 columns
Output: Possible solution [1, 3, 5, 7, 9, 2, 4, 6]
Explanation: Adapt the algorithm to place 8 queens on an 8×10 board, ensuring no two queens threaten each other.
 - b. 5×5 Board with Obstacles:
Input: $N = 5$, Obstacles at positions [(2, 2), (4, 4)]
Output: Possible solution [1, 3, 5, 2, 4]
Explanation: Modify the algorithm to avoid placing queens on obstacle positions, ensuring a valid solution that respects the constraints.
 - c. 6×6 Board with Restricted Positions:
Input: $N = 6$, Restricted positions at columns 2 and 4 for the first queen
Output: Possible solution [1, 3, 5, 2, 4, 6]
Explanation: Adjust the algorithm to handle restricted positions, ensuring the queens are placed without conflicts and within allowed columns.

```
main.c
8  #include <stdio.h>
9  #include <stdlib.h>
10
11  int board[20];
12  int rows, cols;
13
14  int safe(int row, int col)
15  {
16      for(int i=1; i<row; i++)
17      {
18          if(board[i] == col ||
19             abs(board[i] - col) == abs(i - row))
20              return 0;
21      }
22      return 1;
23  }
24
25  void solve(int row)
26  {
27      if(row > rows)
28      {
29          printf("Possible Solution: ");
30          for(int i=1; i<=rows; i++)
31              printf("%d ", board[i]);
32          printf("\n");
33          return;
34      }
35
36      for(int col=1; col<=cols; col++)
37      {
38          if(safe(row, col))
39          {
40              board[row] = col;
```

```
Possible Solution: 9 3 10 2 7 5 1 8
Possible Solution: 9 3 10 4 1 5 2 6
Possible Solution: 9 3 10 4 1 8 2 7
Possible Solution: 9 3 10 4 1 8 5 7
Possible Solution: 9 3 10 4 2 8 5 1
Possible Solution: 9 3 10 4 2 8 5 7
Possible Solution: 9 3 10 4 7 5 2 6
Possible Solution: 9 3 10 7 2 8 1 4
Possible Solution: 9 3 10 7 2 8 5 1
Possible Solution: 9 3 10 7 4 1 5 8
Possible Solution: 9 3 10 7 4 2 5 8
```

3. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

Example 1:

Input: board =

```
[["5","3",".",".","7",".",".","."],
["6",".",".","1","9","5",".","."],
```

```

[".", "9", "8", ".", ".", ".", ".", "6", "."],
["8", ".", ".", ".", "6", ".", ".", ".", "3"],
["4", ".", ".", "8", ".", "3", ".", ".", "1"],
["7", ".", ".", ".", "2", ".", ".", ".", "6"],
[".", "6", ".", ".", ".", ".", "2", "8", "."],
[".", ".", ".", "4", "1", "9", ".", ".", "5"],
[".", ".", ".", ".", "8", ".", ".", "7", "9"]

```

Output:

```

[["5", "3", "4", "6", "7", "8", "9", "1", "2"],
["6", "7", "2", "1", "9", "5", "3", "4", "8"],
["1", "9", "8", "3", "4", "2", "5", "6", "7"],
["8", "5", "9", "7", "6", "1", "4", "2", "3"],
["4", "2", "6", "8", "5", "3", "7", "9", "1"],
["7", "1", "3", "9", "2", "4", "8", "5", "6"],
["9", "6", "1", "5", "3", "7", "2", "8", "4"],
["2", "8", "7", "4", "1", "9", "6", "3", "5"],
["3", "4", "5", "2", "8", "6", "1", "7", "9"]]

```

```

9  #include <stdio.h>
10
11 #define N 9
12
13 int grid[N][N];
14
15 int isSafe(int row, int col, int num)
16 {
17     int x, y;
18
19     for(x = 0; x < N; x++)
20     {
21         if(grid[row][x] == num)
22             return 0;
23     }
24
25     for(x = 0; x < N; x++)
26     {
27         if(grid[x][col] == num)
28             return 0;
29     }
30
31     int startRow = row - row % 3;
32     int startCol = col - col % 3;
33
34     for(x = 0; x < 3; x++)
35     {
36         for(y = 0; y < 3; y++)
37         {
38             if(grid[startRow + x][startCol + y] == num)
39                 return 0;
40         }
41     }

```

input

```
input
Enter Sudoku Grid (use 0 for empty cells):
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

4. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

Example 1:

Input: board =

```
[["5","3",".",".","7",".",".","."],
["6",".",".","1","9","5",".","."],
[["9","8",".",".",".","6","."],
["8",".",".","6",".","."3"],
["4",".","8","3",".","1"],
["7",".","2",".","."6"],
[["6",".","."2","8","."],
[["4","1","9","."5"],
[["8","."7","9"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],
["6","7","2","1","9","5","3","4","8"],
["1","9","8","3","4","2","5","6","7"],
["8","5","9","7","6","1","4","2","3"],
["4","2","6","8","5","3","7","9","1"],
["7","1","3","9","2","4","8","5","6"],
["9","6","1","5","3","7","2","8","4"],
["2","8","7","4","1","9","6","3","5"],
["3","4","5","2","8","6","1","7","9"]]
```

```

9  #include <stdio.h>
10
11  #define N 9
12
13  int grid[N][N];
14
15  int isSafe(int row, int col, int num)
16  {
17      int i, j;
18
19      for(i = 0; i < N; i++)
20      {
21          if(grid[row][i] == num)
22              return 0;
23      }
24
25      for(i = 0; i < N; i++)
26      {
27          if(grid[i][col] == num)
28              return 0;
29      }
30
31      int startRow = row - row % 3;
32      int startCol = col - col % 3;
33
34      for(i = 0; i < 3; i++)
35      {
36          for(j = 0; j < 3; j++)
37          {
38              if(grid[startRow + i][startCol + j] == num)
39                  return 0;
40          }
41      }
42

```

```

Enter Sudoku Grid (0 for empty cells):
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9

```

5. You are given an integer array `nums` and an integer `target`. You want to build an expression out of `nums` by adding one of the symbols `'+'` and `'-'` before each integer in `nums` and then

concatenate all the integers. For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1". Return the number of different expressions that you can build, which evaluates to target.

Example 1:

Input: `nums = [1,1,1,1,1]`, `target = 3`

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of `nums` be target 3.

-1 + 1 + 1 + 1 + 1 = 3

+1 - 1 + 1 + 1 + 1 = 3

+1 + 1 - 1 + 1 + 1 = 3

+1 + 1 + 1 - 1 + 1 = 3

+1 + 1 + 1 + 1 - 1 = 3

Example 2:

Input: `nums = [1]`, `target = 1`

Output: 1

```
8  #include <stdio.h>
9
10 int count = 0;
11
12 void findWays(int nums[], int n, int index, int sum, int target)
13 {
14     if(index == n)
15     {
16         if(sum == target)
17             count++;
18
19         return;
20     }
21
22     findWays(nums, n, index + 1, sum + nums[index], target);
23
24     findWays(nums, n, index + 1, sum - nums[index], target);
25 }
26
27 int main()
28 {
29     int nums[20], n, target;
30
31     printf("Enter number of elements: ");
32     scanf("%d", &n);
33
34     printf("Enter array elements: ");
35     for(int i = 0; i < n; i++)
36         scanf("%d", &nums[i]);
37
38     printf("Enter target: ");
39     scanf("%d", &target);
40 }
```



```
Enter number of elements: 4
Enter array elements: 2 8 6 3
Enter target: 2
Number of ways = 0

...Program finished with exit code 0
Press ENTER to exit console.
```

6. Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo $10^9 + 7$.

Example 1:

Input: arr = [3,1,2,4]

Output: 17

Explanation:

Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].

Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.

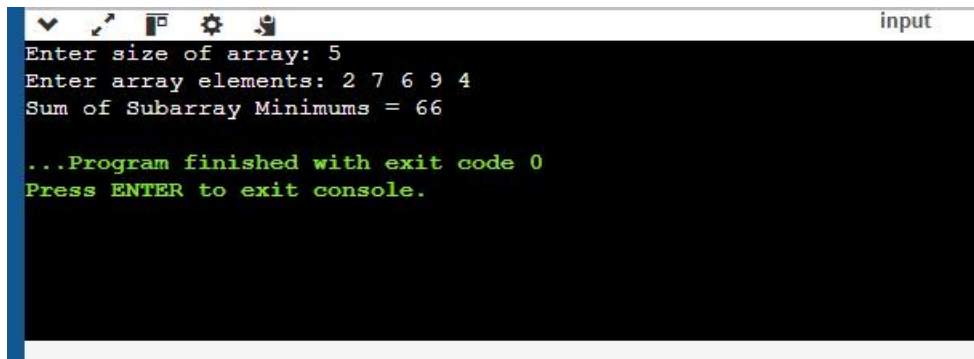
Sum is 17.

Example 2:

Input: arr = [11,81,94,43,3]

Output: 444

```
1  // C++ program to find the sum of minimums of all subarrays
2
3
4
5
6
7
8  #include <stdio.h>
9
10 int main()
11 {
12     int n;
13
14     printf("Enter size of array: ");
15     scanf("%d", &n);
16
17     int arr[n];
18
19     printf("Enter array elements: ");
20     for(int i = 0; i < n; i++)
21         scanf("%d", &arr[i]);
22
23     long long sum = 0;
24
25     for(int i = 0; i < n; i++)
26     {
27         int min;
28
29         for(int j = i; j < n; j++)
30         {
31             if(j == i)
32                 min = arr[j];
33
34             if(arr[j] < min)
35                 min = arr[j];
36
37             sum += min;
38         }
39     }
40 }
```

```
input
Enter size of array: 5
Enter array elements: 2 7 6 9 4
Sum of Subarray Minimums = 66

...Program finished with exit code 0
Press ENTER to exit console.
```

7. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order. The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

```

main.c
8  #include <stdio.h>
9
10 void combinationSum(int arr[], int n, int target, int index,
11                     int result[], int size)
12 {
13     if(target == 0)
14     {
15         printf("[");
16         for(int i = 0; i < size; i++)
17         {
18             printf("%d", result[i]);
19
20             if(i < size - 1)
21                 printf(",");
22         }
23         printf("]\n");
24         return;
25     }
26
27     if(index >= n || target < 0)
28         return;
29
30     result[size] = arr[index];
31
32     combinationSum(arr, n, target - arr[index],
33                   index, result, size + 1);
34
35     combinationSum(arr, n, target,
36                   index + 1, result, size);
37 }
38
39 int main()
40 {

```

```

input
Enter number of candidates: 5
Enter candidate elements: 5 5 4 9 3
Enter target: 8

Possible Combinations:
[5,3]
[5,3]
[4,4]

//www.onlinegdb.com/online_c_compiler#tab-st

```

4. COMBINATION SUM 2:

8. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.

Example 1:

Input: candidates = [10,1,2,7,6,1,5], target = 8

Output:

[[1,1,6]

,

[1,2,5],

[1,7],

[2,6]

]

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

[[1,2,2]

,

[5]

]

```
main.c
7
8 #include <stdio.h>
9 #include <stdlib.h>
10
11 void printCombination(int arr[], int size)
12 {
13     printf("[");
14     for(int i = 0; i < size; i++)
15     {
16         printf("%d", arr[i]);
17
18         if(i < size - 1)
19             printf(",");
20     }
21     printf("]\n");
22 }
23
24 void combinationSum2(int candidates[], int n, int target,
25                     int start, int temp[],
26                     int size)
27 {
28     if(target == 0)
29     {
30         printCombination(temp, size);
31         return;
32     }
33
34     for(int i = start; i < n; i++)
35     {
36         if(i > start &&
37            candidates[i] == candidates[i - 1])
38             continue;
39
40         if(candidates[i] > target)
```

```
input
Enter number of candidates: 4
Enter candidate elements: 1 9 8 2
Enter target: 2

Unique Combinations:
[2]

...Program finished with exit code 0
Press ENTER to exit console.
```

9. Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2:

Input: `nums = [0,1]`

Output: `[[0,1],[1,0]]`

Example 3:

Input: `nums = [1]`

Output: `[[1]]`

```
main.c
8  #include <stdio.h>
9
10 void swap(int *a, int *b)
11 {
12     int temp;
13     temp = *a;
14     *a = *b;
15     *b = temp;
16 }
17
18 void permute(int arr[], int start, int end)
19 {
20     if(start == end)
21     {
22         printf("[");
23
24         for(int i = 0; i <= end; i++)
25         {
26             printf("%d", arr[i]);
27
28             if(i < end)
29                 printf(",");
30         }
31
32         printf("]\n");
33         return;
34     }
35
36     for(int i = start; i <= end; i++)
37     {
38         swap(&arr[start], &arr[i]);
39
40         permute(arr, start + 1, end);
```

```
Enter number of elements: 5
Enter elements: 7 9 6 4 2
```

Permutations are:

```
[7,9,6,4,2]
[7,9,6,2,4]
[7,9,4,6,2]
[7,9,4,2,6]
[7,9,2,4,6]
[7,9,2,6,4]
[7,6,9,4,2]
```

10. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.

Example 1:

Input: nums = [1,1,2]

Output:

```
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

Example 2:

Input: nums = [1,2,3]

Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

```
main.c
8  #include <stdio.h>
9
10 void swap(int *a, int *b)
11 {
12     int temp = *a;
13     *a = *b;
14     *b = temp;
15 }
16
17 int isDuplicate(int arr[], int start, int current)
18 {
19     for(int i = start; i < current; i++)
20     {
21         if(arr[i] == arr[current])
22             return 1;
23     }
24     return 0;
25 }
26
27 void uniquePermutations(int arr[], int start, int end)
28 {
29     if(start == end)
30     {
31         printf("[");
32
33         for(int i = 0; i <= end; i++)
34         {
35             printf("%d", arr[i]);
36
37             if(i < end)
38                 printf(",");
39         }
40     }
```

```
input
Enter number of elements: 5
Enter elements: 7 9 4 6 2

Unique Permutations:
[7,9,4,6,2]
[7,9,4,2,6]
[7,9,6,4,2]
[7,9,6,2,4]
[7,9,2,6,4]
[7,9,2,4,6]
[7,4,9,6,2]
```